

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

I Hemalatha.C (192524475) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

EXPERIMENTS

Q1. You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$$\begin{aligned} E \rightarrow E + T & \quad \{ E.val = E1.val + T.val \} \\ E \rightarrow E - T & \quad \{ E.val = E1.val - T.val \} \\ E \rightarrow T & \quad \{ E.val = T.val \} \end{aligned}$$
$$\begin{aligned} T \rightarrow T * F & \quad \{ T.val = T1.val * F.val \} \\ T \rightarrow T / F & \quad \{ T.val = T1.val / F.val \} \\ T \rightarrow F & \quad \{ T.val = F.val \} \end{aligned}$$
$$\begin{aligned} F \rightarrow (E) & \quad \{ F.val = E.val \} \\ F \rightarrow \text{digit} & \quad \{ F.val = \text{digit} \} \end{aligned}$$

SDD Grammar

The grammar uses **synthesized attributes**. Each child computes its value, and the value is passed upward to the parent.

Production	Semantic Rule
$E \rightarrow E + T$	$E.val = E1.val + T.val$

$E \rightarrow E - T$	$E.val = E1.val - T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T * F$	$T.val = T1.val * F.val$
$T \rightarrow T / F$	$T.val = T1.val / F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit$

Example

Expression:

$2 + 3 * 4$

Evaluation follows precedence:

$3 * 4 = 12$

$2 + 12 = 14$

Final result = 14

The synthesized attribute `.val` moves the computed value from children to their parent nodes.

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

- (i) Evaluates a postfix expression
- (ii) Demonstrates S-attributed definition evaluation
- (iii) Shows how intermediate results are computed

```
#include <stdio.h>

#include <ctype.h>

int stack[100];

int top = -1;

void push(int value)
{
    stack[++top] = value;
}

int pop()
{
    return stack[top--];
}

int main()
{
    char postfix[100];

    int i, a, b, result;

    printf("Enter postfix expression: ");

    scanf("%s", postfix);

    for (i = 0; postfix[i] != '\0'; i++)
    {
        if (isdigit(postfix[i]))
```

```

{
    push(postfix[i] - '0');
}
else
{
    b = pop();
    a = pop();
    switch (postfix[i])
    {
        case '+':
            result = a + b;
            break;
        case '-':
            result = a - b;
            break;
        case '*':
            result = a * b;
            break;
        case '/':
            result = a / b;
            break;
    }
    printf("Intermediate result: %d\n", result);
    push(result);
}
}

```

```
    printf("Final result: %d\n", pop());  
    return 0;  
}
```

Sample Output

Enter postfix expression: 234*+

Intermediate result: 12

Intermediate result: 14

Final result: 14

Explanation

In an **S-attributed definition**, only synthesized attributes are used. In postfix evaluation:

- Operands are pushed onto the stack.
- Operators pop the required operands.
- The operation is performed.
- The intermediate result is pushed back.
- Finally, the stack contains the result.

For 234*+:

$$3 * 4 = 12$$

$$2 + 12 = 14$$

Q3.You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

- (i).Evaluates a simple expression like $a + b * c$
- (ii) Uses function parameters to simulate inherited attributes
- (iii). Ensures correct precedence handling
- (iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

```
#include <stdio.h>

int a, b, c;

int evaluateT()
{
    return b * c;
}

int evaluateE(int inherited)
{
    int t;

    t = evaluateT();

    return inherited + t;
}

int main()
{
    int result;

    printf("Expression: a + b * c\n");

    printf("Enter value of a: ");

    scanf("%d", &a);

    printf("Enter value of b: ");

    scanf("%d", &b);

    printf("Enter value of c: ");

    scanf("%d", &c);

    result = evaluateE(a);
```

```
printf("Result = %d\n", result);

return 0;

}
```

Sample Output

Expression: $a + b * c$

Enter value of a: 2

Enter value of b: 3

Enter value of c: 4

Result = 14

Explanation

Expression:

$a + b * c$

Because $*$ has higher precedence:

$b * c = 3 * 4 = 12$

$a + 12 = 14$

Here, a is passed as a function parameter to `evaluateE()`. This simulates an inherited attribute, where information is passed from the parent to the child during top-down evaluation.

Q4. In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:

(i). Accepts two type expressions (e.g., `int`, `float`, `int*`)

(ii). Compares them for **type equivalence**

(iii). Displays whether they are:

(a). Equivalent

(b). Not equivalent

Extend the logic to handle pointer types (`int*`, `float*`).

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int typeEquivalent(char type1[], char type2[])
```

```
{
```

```
    return strcmp(type1, type2) == 0;
```

```
}
```

```
int main()
```

```
{
```

```
    char type1[20], type2[20];
```

```
    printf("Enter first type expression: ");
```

```
    scanf("%s", type1);
```

```
    printf("Enter second type expression: ");
```

```
    scanf("%s", type2);
```

```
    if (typeEquivalent(type1, type2))
```

```
        printf("Equivalent\n");
```

```
    else
```

```
        printf("Not equivalent\n");
```

```
    return 0; }
```

Sample Output 1

Enter first type expression: `int*`

Enter second type expression: `int*`

Equivalent

Sample Output 2

Enter first type expression: `int*`

Enter second type expression: `float*`

Not equivalent

Explanation

The program compares the complete type expression.

Examples:

`int` = `int` → Equivalent

`float` = `int` → Not equivalent

`int*` = `int*` → Equivalent

`int*` = `float*` → Not equivalent

Thus, pointer types are also handled because `int*` and `float*` are treated as different type expressions.

Q5. You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

- (i). Accepts two operands and their types
- (ii). Checks whether the operation (e.g., +, *) is valid
- (iii). Reports:
 - (a). Valid expression
 - (b). Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int validOperation(char type1[], char type2[], char op)
```

```
{  
    if (strcmp(type1, "int") == 0 &&  
        strcmp(type2, "int") == 0)  
    {  
        return 1;  
    }  
    if (strcmp(type1, "float") == 0 &&  
        strcmp(type2, "float") == 0)  
    {  
        return 1;  
    }  
    if (strcmp(type1, "int") == 0 &&  
        strcmp(type2, "float") == 0)  
    {  
        return 1;  
    }  
    if (strcmp(type1, "float") == 0 &&  
        strcmp(type2, "int") == 0)
```

```

    {
        return 1;
    }
    return 0;
}
int main()
{
    char type1[20], type2[20], op;
    printf("Enter first operand type: ");
    scanf("%s", type1);
    printf("Enter operator (+ or *): ");
    scanf(" %c", &op);
    printf("Enter second operand type: ");
    scanf("%s", type2);
    if (validOperation(type1, type2, op))
        printf("Valid expression\n");
    else
        printf("Type error\n");
    return 0;
}

```

Sample Output 1

Enter first operand type: int

Enter operator (+ or *): +

Enter second operand type: int

Valid expression

Sample Output 2

Enter first operand type: int

Enter operator: +

Enter second operand type: char*

Type error

Explanation

The **semantic analyzer** checks whether the operands have compatible types before generating code.

For example:

int + int → Valid

int * float → Valid

float + int → Valid

int + char* → Type error

float * char* → Type error

This prevents invalid operations from being passed to later compiler phases.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

